

GUIA TÉCNICO DESCRITIVO DE VULNERABILIDADES EM CÓDIGO GERADO POR INTELIGÊNCIA ARTIFICIAL

Este documento apresenta o guia técnico de mitigação de vulnerabilidades em código gerado por inteligência artificial, desenvolvido como artefato complementar ao Trabalho de Conclusão de Curso "Mitigação de Vulnerabilidades de Segurança em Código Gerado por Inteligência Artificial" (UFSM, 2026). O guia organiza sete categorias de vulnerabilidades recorrentes em código gerado por modelos de linguagem, estruturando para cada uma delas: identificação, descrição, evidências na literatura, mapeamento para taxonomias CWE e OWASP, exemplos de ocorrência, impacto potencial, forma de detecção, recomendações de mitigação e prioridade.

GIA-001 – VALIDAÇÃO INADEQUADA DE ENTRADA E INJEÇÃO

Identificador interno: GIA-001

Categoria: Validação inadequada de entrada e injeção

Descrição: Categoria que reúne falhas nas quais dados externos são utilizados sem validação, normalização, sanitização ou parametrização adequada. Em código gerado por IA, esse problema pode ocorrer quando o modelo produz uma solução funcional, porém omite controles de entrada ou constrói comandos, consultas e respostas HTML por concatenação de strings.

Evidências na literatura: Pearce et al. (2022) e Fu et al. (2025) relatam vulnerabilidades recorrentes em código gerado ou assistido por IA relacionadas a validação inadequada e injeção. Dora et al. (2025) também identificam problemas desse tipo em aplicações web geradas por LLMs.

Mapeamento CWE/OWASP: CWE-20 (<https://cwe.mitre.org/data/definitions/20.html>), CWE-78 (<https://cwe.mitre.org/data/definitions/78.html>), CWE-79 (<https://cwe.mitre.org/data/definitions/79.html>), CWE-89 (<https://cwe.mitre.org/data/definitions/89.html>); OWASP A03:2021 – Injection (https://owasp.org/Top10/A03_2021-Injection/).

Exemplos de ocorrência: Concatenação de entrada do usuário em consultas SQL; uso de parâmetros externos em chamadas de sistema; renderização de dados não tratados em páginas web; uso de eval, exec ou funções equivalentes com conteúdo controlável pelo usuário.

Impacto potencial: Pode permitir execução de comandos, acesso indevido a dados, manipulação de consultas, Cross-Site Scripting, comprometimento da aplicação e movimentação para outros componentes do sistema.

Forma de detecção: SAST para padrões conhecidos de injeção; heurísticas para identificar concatenação de strings com dados externos; revisão apoiada em trechos que recebem input, request, args, form, query, headers ou payloads e os utilizam em comandos, consultas ou renderização.

Recomendações de mitigação:

O que verificar: identificar no código todos os pontos em que dados externos — provenientes de input, request, args, form, headers ou fontes equivalentes — são utilizados em consultas SQL, chamadas de sistema operacional, funções de execução dinâmica (eval, exec) ou renderizados diretamente em páginas HTML sem tratamento prévio (OWASP FOUNDATION, [s. d.]).

Como corrigir:

Consultas SQL: substituir qualquer construção que concatene o valor externo diretamente na string da consulta por consultas parametrizadas. Padrão inseguro:

```
query = "SELECT * FROM users WHERE name = '" + nome + "'"
```

Padrão seguro:

```
cursor.execute("SELECT * FROM users WHERE name = ?", (nome,))
```

Chamadas de sistema operacional: substituir chamadas com shell=True pela forma com lista de argumentos e shell=False. Padrão inseguro:

```
subprocess.run("ls " + pasta, shell=True)
```

Padrão seguro:

```
subprocess.run(["ls", pasta], shell=False)
```

Saídas renderizadas em HTML: aplicar codificação de saída sensível ao contexto.

Padrão seguro (framework com auto-escape):

```
return render_template("perfil.html", nome=nome)
```

Validação por lista de permissões: verificar se o valor pertence ao conjunto esperado.

```
ORDENACOES_VALIDAS = {"asc", "desc"}; if ordenacao not in  
ORDENACOES_VALIDAS: raise ValueError("Ordenação inválida")
```

Funções de execução dinâmica: eliminar eval/exec com conteúdo externo. Para cálculos matemáticos, usar asteval:

```
from asteval import Interpreter; aeval = Interpreter(); resultado =  
aeval(formula)
```

Como validar: executar análise estática com Bandit (regras B608, B602, B603, B307) e confirmar ausência de novos achados. Complementar com testes funcionais utilizando payloads representativos (' OR '1'='1 para SQL; ; cat /etc/passwd para comandos; <script>alert(1)</script> para XSS). Verificar o requisito ASVS 1.2.5 (OWASP FOUNDATION, 2025).

Prioridade: Alta, devido ao potencial de exploração direta e ao impacto recorrente em aplicações web, scripts e integrações geradas por IA.

GIA-002 – FALHAS DE AUTENTICAÇÃO, AUTORIZAÇÃO E CONTROLE DE ACESSO

Identificador interno: GIA-002

Categoria: Falhas de autenticação, autorização e controle de acesso

Descrição: Categoria que reúne situações em que o código gerado não verifica adequadamente identidade, sessão, permissões ou escopo de acesso antes de executar uma operação sensível. Em saídas de modelos de linguagem, a funcionalidade principal pode ser implementada, mas os controles de acesso podem ser omitidos, simplificados ou posicionados em local inadequado.

Evidências na literatura: Dora et al. (2025) e Zhao et al. (2025) relatam falhas em aplicações web geradas por LLMs envolvendo autenticação, gerenciamento de sessão, autorização e proteção de recursos. Esses achados indicam que a ausência de contexto do sistema pode favorecer controles incompletos.

Mapeamento CWE/OWASP: OWASP A01:2021 – Broken Access Control (https://owasp.org/Top10/A01_2021-Broken_Access_Control/); OWASP A07:2021 – Identification and Authentication Failures (https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/); CWE-306 (<https://cwe.mitre.org/data/definitions/306.html>), CWE-862

(<https://cwe.mitre.org/data/definitions/862.html>),
(<https://cwe.mitre.org/data/definitions/863.html>).

CWE-863

Exemplos de ocorrência: Rota administrativa sem verificação de sessão; endpoint que altera dados sem checar o proprietário do recurso; uso de tokens sem validação adequada; funções sensíveis acessíveis diretamente por identificador previsível.

Impacto potencial: Pode resultar em acesso não autorizado, alteração ou exclusão indevida de dados, exposição de informações sensíveis, escalonamento de privilégios e comprometimento de funcionalidades críticas.

Forma de detecção: Heurísticas para localizar rotas ou funções sensíveis sem chamadas de autenticação/autorização; revisão apoiada de endpoints com verbos de alteração; análise de uso de decorators, middlewares, checagem de sessão, roles e ownership.

Recomendações de mitigação:

O que verificar: identificar rotas e funções que executam operações sensíveis — leitura, alteração, exclusão, acesso administrativo — e verificar se cada uma exige autenticação e autorização explícitas. Verificar também se endpoints recebem identificadores de recurso sem checar propriedade (OWASP FOUNDATION, [s. d.]).

Como corrigir:

Autenticação ausente: toda rota sensível deve exigir autenticação explícita via mecanismo consolidado do framework. Padrão inseguro:

```
@app.route("/admin/usuarios") def listar_usuarios(): return
Usuario.query.all()
```

Padrão seguro:

```
@app.route("/admin/usuarios") @login_required def listar_usuarios(): return
Usuario.query.all()
```

Autorização ausente (IDOR): verificar no servidor se o recurso pertence ao usuário autenticado. Padrão seguro:

```
pedido = Pedido.query.filter_by(id=pedido_id,
usuario_id=current_user.id).first_or_404()
```

Autorização por papel: verificação deve ocorrer no servidor via decorator. Exemplo:

```
@roles_required("admin") def deletar_usuario(id): ...
```

Princípio do menor privilégio e negação por padrão: se não houver regra explícita permitindo, a operação é bloqueada (OWASP FOUNDATION, [s. d.]).

Como validar: testes para acesso sem autenticação (espera HTTP 401); acesso com papel insuficiente (espera HTTP 403); acesso cruzado entre usuários A→B pelo ID (espera HTTP 403 ou 404). Verificar requisitos ASVS de controle de acesso (OWASP FOUNDATION, 2025).

Prioridade: Alta, pois a ausência de controle de acesso pode permitir exploração direta mesmo quando o restante da aplicação aparenta funcionar corretamente.

GIA-003 – USO INSEGURO DE BIBLIOTECAS, APIS E DEPENDÊNCIAS

Identificador interno: GIA-003

Categoria: Uso inseguro de bibliotecas, APIs e dependências

Descrição: Categoria relacionada ao uso de bibliotecas, APIs, pacotes ou dependências de forma insegura, desatualizada, inexistente ou não verificada. No contexto de IA generativa, inclui o risco de package hallucination, em que o modelo sugere nomes de pacotes plausíveis, porém inexistentes ou incorretos.

Evidências na literatura: Spracklen et al. (2025) discutem riscos de package hallucination em código gerado por modelos de linguagem, com taxas de alucinação de 5,2% a 21,7% dependendo do modelo. A literatura sobre código gerado por IA também indica que modelos podem sugerir bibliotecas ou APIs sem garantir procedência, versão, manutenção ou adequação de segurança.

Mapeamento CWE/OWASP: CWE-829 (<https://cwe.mitre.org/data/definitions/829.html>), CWE-1104 (<https://cwe.mitre.org/data/definitions/1104.html>); OWASP A06:2021 – Vulnerable and Outdated Components (https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/); riscos de cadeia de suprimentos.

Exemplos de ocorrência: Importação de pacote inexistente; dependência com nome semelhante a pacote legítimo (slopsquatting); uso de versão vulnerável (ex.: lxml 4.6.0, jinja2 3.1.1); instalação automática de pacote sugerido por IA sem verificação; chamada de API depreciada ou insegura.

Impacto potencial: Pode permitir ataques de cadeia de suprimentos, instalação de pacotes maliciosos, exploração de vulnerabilidades conhecidas, quebra de compatibilidade e incorporação de código externo não confiável.

Forma de detecção: Heurísticas para extrair dependências e verificar existência, origem, versão e reputação; comparação com arquivos de dependências; SAST para chamadas de APIs inseguras; pip-audit para vulnerabilidades conhecidas em dependências Python.

Recomendações de mitigação:

O que verificar: identificar todas as importações e comandos de instalação no código gerado por IA. Spracklen et al. (2025) demonstram taxas de alucinação de 5,2% (modelos comerciais) a 21,7% (modelos abertos), com mais de 205 mil nomes únicos inventados. Uma replicação de 2026 confirmou taxas de 4,62% a 6,10% nos modelos mais recentes. O risco concreto é o slopsquatting: atacante registra o nome alucinado com código malicioso. Caso real: huggingface-cli acumulou >30.000 downloads antes de ser identificada (SPRACKLEN et al., 2025).

Como corrigir:

Verificar existência e origem antes de instalar: confirmar o nome exato em <https://pypi.org>, verificar mantenedor, data da última versão, repositório e volume de downloads em <https://pypistats.org/packages/<nome>>. Caso do colourama (variante maliciosa de colorama que roubava Bitcoin) e ultralytics (YOLO, ~80M downloads/mês, comprometida em dez/2024 com minerador em 4 versões).

Fixar versões e usar hashes. Exemplo de requirements.txt correto:

```
Django==5.2.1 requests==2.33.0 lxml==4.9.4 jinja2==3.1.4
```

Instalar com verificação de hash:

```
pip install -r requirements.txt --require-hashes
```

Verificar vulnerabilidades com pip-audit (Trail of Bits + Google, escaneia contra PyPI Advisory Database e OSV):

```
pip install pip-audit && pip-audit -r requirements.txt
```

Exemplo de saída: lxml 4.6.0 / PYSEC-2021-19 / Fix: 4.6.3

Corrigir automaticamente: pip-audit -r requirements.txt --fix

Integração CI GitHub Actions: uses: pypa/gh-action-pip-audit@v1.0.0 with inputs: requirements.txt

Como validar: `pip-audit -r requirements.txt` no CI sem achados. `requirements.txt` com versões exatas commitado. Cada dependência com página oficial ativa no PyPI. Para rastreabilidade: `pip-audit -r requirements.txt -f cyclonedx-json` (OWASP FOUNDATION, [s. d.]).

Prioridade: Alta, especialmente quando o código gerado inclui comandos de instalação, importações novas ou dependências não conhecidas pelo projeto.

GIA-004 – EXPOSIÇÃO DE DADOS SENSÍVEIS E SEGREDOS NO CÓDIGO

Identificador interno: GIA-004

Categoria: Exposição de dados sensíveis e segredos no código

Descrição: Categoria que abrange a exposição de credenciais, chaves, tokens, senhas, dados pessoais, caminhos internos ou informações sensíveis diretamente no código, em logs ou em respostas da aplicação. Em código gerado por IA, pode ocorrer quando o modelo utiliza valores fictícios de forma insegura ou sugere padrões que acabam sendo incorporados como implementação real.

Evidências na literatura: Pearce et al. (2022), Fu et al. (2025) e Tihanyi et al. (2025) indicam que códigos gerados por IA podem conter práticas inseguras relacionadas à proteção inadequada de informações e ao uso de valores sensíveis diretamente no código.

Mapeamento CWE/OWASP: CWE-798 (<https://cwe.mitre.org/data/definitions/798.html>), CWE-200 (<https://cwe.mitre.org/data/definitions/200.html>); OWASP A02:2021 – Cryptographic Failures (https://owasp.org/Top10/A02_2021-Cryptographic_Failures/).

Exemplos de ocorrência: Chave de API codificada no código-fonte; senha em string literal; token impresso em log; retorno de erro contendo segredo; credenciais em arquivo de exemplo sem indicação clara de substituição por variável de ambiente.

Impacto potencial: Pode permitir acesso indevido a serviços, vazamento de dados, comprometimento de contas, uso abusivo de APIs e exposição de informações internas da aplicação ou da infraestrutura.

Forma de detecção: SAST para padrões de hardcoded secrets; heurística H009 para segredos identificados pelo formato do valor (padrões como `sk_live_`, `AKIA` e similares); heurísticas baseadas em nomes de variáveis; revisão apoiada de logs, mensagens de erro, arquivos de configuração e exemplos gerados.

Recomendações de mitigação:

O que verificar: identificar todas as strings com padrão de chave, token, senha ou credencial (atribuições a variáveis como `API_KEY`, `SECRET`, `PASSWORD`, `TOKEN`) e segredos embutidos em URLs de conexão. Verificar logs e mensagens de erro que possam imprimir credenciais ou dados pessoais, e arquivos de exemplo ou de configuração (`.env`, `settings`) que contenham valores reais em vez de placeholders (CWE-798; CWE-200; OWASP FOUNDATION, [s. d.]).

Como corrigir:

Segredos embutidos no código: substituir valores fixos pela leitura de variável de ambiente. Padrão inseguro:

```
API_KEY = "sk-1a2b3c4d5e6f"
```

Padrão seguro:

```
API_KEY = os.environ["API_KEY"]
```

Logs com credenciais: registrar identificadores, nunca o segredo. Padrão inseguro:

```
logging.info(f"login com senha {senha}")
```

Padrão seguro:

```
logging.info("login do usuário %s", usuario)
```

Arquivos de exemplo: usar placeholder e manter o arquivo real fora do versionamento.

Padrão inseguro (`.env` versionado):

```
API_KEY=sk-1a2b3c4d5e6f
```

Padrão seguro (`.env.example`, com `.env` no `.gitignore`):

```
API_KEY=coloque-sua-chave-aqui
```

Rotação e higienização: rotacionar credenciais expostas e limpar o histórico de versionamento quando necessário.

Varredura automatizada: aplicar ferramentas de secret scanning (por exemplo, `gitleaks` ou `detect-secrets`) no repositório e no pipeline.

Como validar: executar uma ferramenta de secret scanning (por exemplo, `gitleaks` ou `detect-secrets`) sobre o repositório e no pipeline de CI, confirmando ausência de

achados. Confirmar que os segredos foram movidos para variáveis de ambiente (os.environ.get) ou cofre e que o SAST Bandit (regras B105, B106, B107) não acusa senhas embutidas. Verificar que as respostas de erro não retornam valores sensíveis e, em caso de exposição prévia, que a credencial foi rotacionada e o histórico higienizado. Verificar o capítulo de Proteção de Dados do OWASP ASVS 5.0 (OWASP FOUNDATION, 2025).

Prioridade: Alta, dado o impacto imediato de credenciais expostas e o risco de uso indevido mesmo em casos de placeholders inseguros.

GIA-005 – TRATAMENTO INADEQUADO DE ERROS E EXCEÇÕES

Identificador interno: GIA-005

Categoria: Tratamento inadequado de erros e exceções

Descrição: Categoria que reúne implementações em que erros são ignorados, tratados de forma genérica, mascarados ou expostos indevidamente. Em código gerado por IA, é comum que a solução priorize o fluxo funcional e utilize blocos de exceção simplificados, como capturas genéricas ou silenciamento de falhas.

Evidências na literatura: Sajadi et al. (2025) e Tihanyi et al. (2025) apontam que respostas e programas gerados por LLMs podem apresentar práticas de segurança superficiais ou incompletas. Esse comportamento favorece tratamento de erros sem registro, sem recuperação segura ou com exposição excessiva de detalhes.

Mapeamento CWE/OWASP: CWE-703 (<https://cwe.mitre.org/data/definitions/703.html>), CWE-209 (<https://cwe.mitre.org/data/definitions/209.html>); OWASP A09:2021 – Security Logging and Monitoring Failures (https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/).

Exemplos de ocorrência: except Exception: pass; retorno de stack trace ao usuário; captura genérica sem log; continuação da execução após falha crítica; mensagens de erro que expõem caminhos internos, consultas, tokens ou detalhes de infraestrutura.

Impacto potencial: Pode ocultar falhas exploráveis, dificultar auditoria, permitir estados inconsistentes, expor informações sensíveis e reduzir a capacidade de detecção de incidentes.

Forma de detecção: Heurísticas para localizar capturas genéricas, blocos vazios, pass em exceções, prints de exceção e respostas contendo detalhes internos; SAST quando houver regras específicas para tratamento inseguro.

Recomendações de mitigação:

O que verificar: localizar blocos de captura genérica e silenciosa (except Exception: pass, except: pass), retorno de stack trace ou de traceback diretamente ao usuário, continuação da execução após falha crítica e mensagens de erro que exponham caminhos internos, consultas, tokens ou detalhes de infraestrutura (CWE-703; CWE-209; OWASP FOUNDATION, [s. d.]).

Como corrigir:

Captura genérica silenciosa: tratar exceção específica, registrar o erro e repropagar quando necessário. Padrão inseguro:

```
try:
    processar(pedido)
except Exception:
    pass
```

Padrão seguro:

```
try:
    processar(pedido)
except PedidoInvalidoError:
    logging.error("Falha ao processar pedido", exc_info=True)
    raise
```

Stack trace ao usuário: retornar mensagem genérica com identificador de correlação e registrar os detalhes apenas no log. Padrão inseguro:

```
return {"error": traceback.format_exc()}, 500
```

Padrão seguro:

```
ref = uuid4().hex
logging.error("Erro %s", ref, exc_info=True)
return {"error": "Erro interno", "ref": ref}, 500
```

Falha segura: em falha crítica, interromper a operação em vez de prosseguir em estado inconsistente.

Como validar: escrever testes dos caminhos de erro que comprovem dois comportamentos: a resposta ao usuário é genérica (sem detalhes técnicos) e a falha é registrada em log seguro (logging.error(..., exc_info=True)) sem dados sensíveis.

Confirmar que o SAST Bandit (regras B110 e B112) e as heurísticas H001/H002 não acusam capturas silenciosas. Verificar o capítulo de Registro de Segurança e Tratamento de Erros do OWASP ASVS 5.0 (OWASP FOUNDATION, 2025).

Prioridade: Média, podendo ser alta quando o erro expõe dados sensíveis, permite bypass de controles ou oculta falhas críticas de segurança.

GIA-006 – CONFIGURAÇÕES INSEGURAS E PROTEÇÕES AUSENTES

Identificador interno: GIA-006

Categoria: Configurações inseguras e proteções ausentes

Descrição: Categoria que abrange configurações frágeis, valores padrão inseguros, ausência de proteções recomendadas e mecanismos de defesa não habilitados. Em aplicações geradas por IA, essas falhas podem surgir quando o modelo produz uma implementação mínima funcional sem configurar adequadamente segurança de sessão, cabeçalhos, criptografia, CORS, debug ou proteção de rotas.

Evidências na literatura: Dora et al. (2025) e Zhao et al. (2025) relatam problemas em aplicações geradas por LLMs envolvendo proteção de recursos, configurações web, autenticação, sessão e mecanismos de segurança ausentes ou incompletos.

Mapeamento CWE/OWASP: OWASP A05:2021 – Security Misconfiguration (https://owasp.org/Top10/A05_2021-Security_Misconfiguration/); CWE-16 (<https://cwe.mitre.org/data/definitions/16.html>).

Exemplos de ocorrência: debug=True em ambiente de produção; CORS permissivo; ausência de cabeçalhos de segurança; cookie sem flags Secure, HttpOnly ou SameSite; endpoint sensível sem rate limit; configurações padrão mantidas sem revisão.

Impacto potencial: Pode ampliar a superfície de ataque, facilitar exploração de falhas, permitir vazamento de informações, enfraquecer sessões e reduzir a resistência da aplicação contra ataques comuns.

Forma de detecção: Heurísticas para configurações de framework, flags inseguras, CORS permissivo, ausência de cabeçalhos e uso de modo debug; revisão apoiada de arquivos de configuração, inicialização da aplicação e middlewares.

Recomendações de mitigação:

O que verificar: verificar se o modo de depuração está desativado em produção (DEBUG=False no Django, debug=False no Flask), se a política de CORS não usa origem curinga (*), se os cookies de sessão possuem as flags Secure, HttpOnly e SameSite, se os cabeçalhos de segurança recomendados estão presentes (Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options) e se endpoints sensíveis possuem limitação de taxa. Revisar configurações padrão de framework mantidas sem ajuste (CWE-16; OWASP FOUNDATION, [s. d.]).

Como corrigir:

Modo de depuração em produção: derivar o valor de variável de ambiente, com padrão seguro. Padrão inseguro:

```
DEBUG = True
```

Padrão seguro:

```
DEBUG = os.environ.get("DEBUG", "False") == "True"
```

CORS permissivo: restringir a origens confiáveis explícitas. Padrão inseguro:

```
CORS(app, origins="*")
```

Padrão seguro:

```
CORS(app, origins=["https://meusite.com"])
```

Cookies sem flags de segurança: habilitar Secure, HttpOnly e SameSite. Padrão inseguro:

```
app.config["SESSION_COOKIE_SECURE"] = False
```

Padrão seguro:

```
app.config.update(SESSION_COOKIE_SECURE=True, SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SAMESITE="Lax")
```

Hardening do framework: aplicar o checklist de implantação do framework adotado e revisar os defaults antes de publicar.

Como validar: executar o checklist de implantação do framework — no Django, python manage.py check --deploy sem alertas (DJANGO SOFTWARE FOUNDATION, [s. d.]). Avaliar os cabeçalhos de resposta com base nas recomendações do OWASP Secure

Headers Project (OWASP FOUNDATION, [s. d.]) e testar a política de CORS rejeitando origem não autorizada. Confirmar que o SAST Bandit (regras B201 e B501 a B506) não acusa configurações inseguras e que os cookies apresentam as flags. Verificar o capítulo de Configuração do OWASP ASVS 5.0 (OWASP FOUNDATION, 2025).

Prioridade: Alta, dado o potencial de ampliar a superfície de ataque e o impacto em aplicações expostas.

GIA-007 – IMPLEMENTAÇÃO INCOMPLETA DE CONTROLES DE SEGURANÇA

Identificador interno: GIA-007

Categoria: Implementação incompleta de controles de segurança

Descrição: Categoria voltada a situações em que o código sugere a existência de um controle de segurança, mas a implementação é parcial, superficial, pendente ou insuficiente. Em código gerado por IA, isso pode ocorrer quando o modelo produz uma solução aparentemente completa, mas deixa validações, verificações ou políticas apenas como comentários, placeholders ou estruturas incompletas.

Evidências na literatura: Sajadi et al. (2025) e Tihanyi et al. (2025) indicam que códigos e respostas de programação produzidos por LLMs podem ser funcionais, porém inseguros ou incompletos do ponto de vista de práticas de segurança.

Mapeamento CWE/OWASP: Mapeamento variável conforme o controle ausente ou incompleto; pode se relacionar a categorias de validação, autenticação, autorização, criptografia, logging, configuração ou tratamento de erro.

Exemplos de ocorrência: Comentário TODO em validação de senha; função `is_authorized` retornando sempre True; placeholder para criptografia; validação implementada apenas no cliente; bloco de segurança citado em comentário, mas ausente no código executável.

Impacto potencial: Pode gerar falsa sensação de segurança, permitir incorporação de código incompleto em produção, causar bypass de controles e dificultar a identificação de falhas durante revisão superficial.

Forma de detecção: Heurísticas para TODO, FIXME, pass, placeholders, retornos constantes em funções de segurança e comentários indicando implementação futura; revisão apoiada de funções com nomes associados a validação, autenticação, autorização, criptografia e verificação.

Recomendações de mitigação:

O que verificar: revisar funções com nomes associados a controles de segurança (validação, autenticação, autorização, criptografia, verificação) em busca de comentários TODO/FIXME, retornos constantes (por exemplo, `def is_authorized(): return True`), placeholders e validações presentes apenas no cliente ou citadas somente em comentários, sem implementação executável (OWASP FOUNDATION, [s. d.]).

Como corrigir:

Função de verificação como stub: substituir o retorno constante por uma verificação real e testável. Padrão inseguro:

```
def is_authorized(user, recurso):  
    # TODO: implementar verificação de permissão  
    return True
```

Padrão seguro:

```
def is_authorized(user, recurso):  
    return recurso.owner_id == user.id or user.has_role("admin")
```

Validação apenas no cliente: replicar a validação no servidor, que é a fronteira de confiança. Padrão inseguro:

```
# validação feita apenas no JavaScript do formulário
```

Padrão seguro:

```
if not re.fullmatch(r"^[^@]+@[^@]+\.[^@]+", email):  
    raise ValueError("E-mail inválido")
```

Bloqueio de pendências: não integrar código com controle explicitamente pendente e criar um teste que prove o comportamento do controle.

Como validar: para cada controle, criar um teste específico que comprove o comportamento esperado, garantindo que o caso negativo realmente falhe (acesso não autorizado deve ser bloqueado; senha inválida deve ser rejeitada). Confirmar a ausência de TODO/FIXME e de placeholders em funções de segurança no diff aprovado e que as heurísticas H003/H004 não acusam controles incompletos. Seguir as práticas de revisão (PW.7) e de teste de código (PW.8) do NIST SP 800-218 (NIST, 2022) e o

capítulo de Codificação Segura e Arquitetura do OWASP ASVS 5.0 (OWASP FOUNDATION, 2025).

Prioridade: Alta, pois controles incompletos podem criar falsa sensação de segurança e permitir incorporação de código inadequado em produção.

REFERÊNCIAS

- CHEN, Mark et al. Evaluating Large Language Models Trained on Code. arXiv, 2021. DOI: 10.48550/arXiv.2107.03374.
- CVE PROGRAM. Common Vulnerabilities and Exposures (CVE). [s. d.]. Disponível em: <https://www.cve.org/>.
- DJANGO SOFTWARE FOUNDATION. Deployment checklist. [s. d.]. Disponível em: <https://docs.djangoproject.com/en/stable/howto/deployment/checklist/>.
- DORA, Swaroop et al. The Hidden Risks of LLM-Generated Web Application Code. arXiv, 2025. DOI: 10.48550/arXiv.2504.20612.
- FU, Yujia et al. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. ACM Transactions on Software Engineering and Methodology, v. 34, n. 8, 2025. DOI: 10.1145/3716848.
- HE, Jingxuan; VECHEV, Martin. Large Language Models for Code: Security Hardening and Adversarial Testing. In: ACM CCS 2023. New York: ACM, 2023. p. 1865-1879. DOI: 10.1145/3576915.3623175.
- HE, Jingxuan et al. Instruction Tuning for Secure Code Generation. In: ICML 2024. PMLR, v. 235, 2024. p. 18043-18062. Disponível em: <https://proceedings.mlr.press/v235/he24k.html>.
- LIU, Yue et al. On the Reliability and Explainability of Language Models for Program Generation. ACM Transactions on Software Engineering and Methodology, v. 33, n. 5, 2024. DOI: 10.1145/3641540.
- MITRE. Common Weakness Enumeration (CWE). [s. d.]. Disponível em: <https://cwe.mitre.org/>.
- NIST. Secure Software Development Framework (SSDF) Version 1.1. NIST SP 800-218. Gaithersburg: NIST, 2022. DOI: 10.6028/NIST.SP.800-218.

- NIST. Secure Software Development Practices for Generative AI and Dual-Use Foundation Models. NIST SP 800-218A. Gaithersburg: NIST, 2024. DOI: 10.6028/NIST.SP.800-218A.
- NIST. National Vulnerability Database (NVD). [s. d.]. Disponível em: <https://nvd.nist.gov/>.
- OWASP FOUNDATION. OWASP Application Security Verification Standard (ASVS), version 5.0.0. [S. l.], 2025. Disponível em: <https://owasp.org/www-project-application-security-verification-standard/>.
- OWASP FOUNDATION. OWASP Cheat Sheet Series. [S. l.], [s. d.]. Disponível em: <https://cheatsheetseries.owasp.org/>.
- OWASP FOUNDATION. OWASP Secure Headers Project. [S. l.], [s. d.]. Disponível em: <https://owasp.org/www-project-secure-headers/>.
- OWASP FOUNDATION. OWASP Top 10: The Ten Most Critical Web Application Security Risks. 2021. Disponível em: <https://owasp.org/www-project-top-ten/>.
- PEARCE, Hammond et al. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In: IEEE S&P 2022. Los Alamitos: IEEE, 2022. p. 754-768. DOI: 10.1109/SP46214.2022.9833571.
- PEARCE, Hammond et al. Examining Zero-Shot Vulnerability Repair with Large Language Models. In: IEEE S&P 2023. San Francisco: IEEE, 2023. p. 2339-2356. DOI: 10.1109/SP46215.2023.10179324.
- SAJADI, Amirali et al. Do LLMs consider security? An empirical study on responses to programming questions. Empirical Software Engineering, v. 30, 2025. DOI: 10.1007/s10664-025-10658-6.
- SANDOVAL, Gustavo et al. Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. In: USENIX Security 23. USENIX Association, 2023. p. 2205-2222.
- SPRACKLEN, Joseph et al. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In: 34th USENIX Security Symposium. Berkeley: USENIX Association, 2025. p. 3687-3706.

TIHANYI, Norbert et al. How secure is AI-generated code: a large-scale comparison of large language models. *Empirical Software Engineering*, v. 30, n. 2, 2025. DOI: 10.1007/s10664-024-10590-1.

WADHAMS, Zachary D. et al. Barriers to Using Static Application Security Testing (SAST) Tools. In: *IEEE/ACM HCSE&CS 2024*. New York: ACM, 2024. DOI: 10.1145/3691621.3694947.

XIE, Ying. VibeGuard: A Security Gate Framework for AI-Generated Code. *arXiv*, 2026. DOI: 10.48550/arXiv.2604.01052.

ZHAO, Songwen et al. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. *arXiv*, 2025. DOI: 10.48550/arXiv.2512.03262.